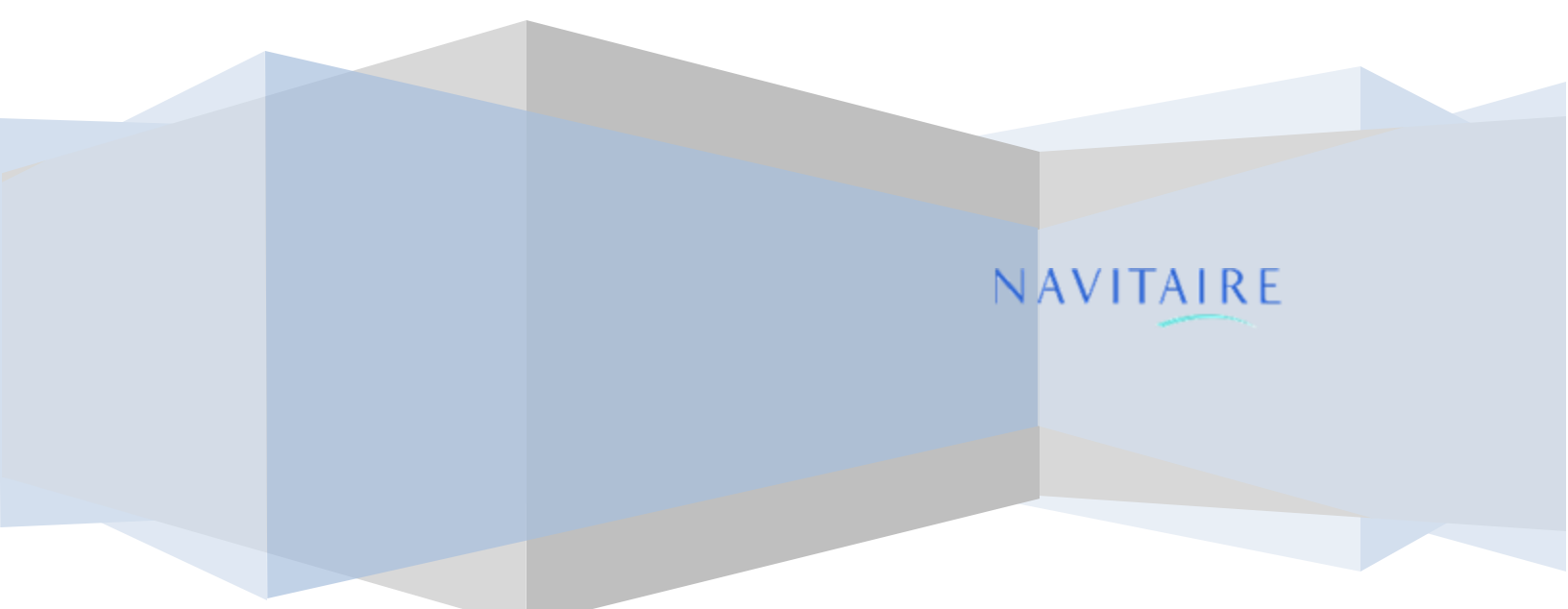




Technology that liberates

API Development Best Practices

The bottom half of the page features a large, abstract geometric design composed of several overlapping, semi-transparent polygons in shades of light blue and grey. The NAVITAIRE logo is positioned on the right side of this design, appearing as if it is part of the background structure.

NAVITAIRE

Table of Contents

Introduction.....2

 Purpose.....2

 Session Management.....3

 Session Pooling3

 Session Timeout.....3

 HTTP Compression.....4

 Enabling Compression4

 Filtered Methods.....6

 BookingManager.....6

 Create Booking Flow7

 Logon7

 Availability.....7

 Price Itinerary7

 Sell (Journey)7

 UpdatePassengers.....7

 AssignSeats.....8

 Sell (SSR).....8

 UpdateContacts8

 AddPaymentToBooking8

 BookingCommit.....8

 Logout.....8

Errors returned by API method.....8

Introduction

Purpose

This document describes the recommended approach to API development using the New Skies SOAP web services.

Session Management

Since New Skies requires a session for all API interactions, proper session management is paramount to performance. Applications should re-use sessions for repetitive functions and explicitly call **Logout** at the end of processes to dispose of the session and free app server memory.

Session Pooling

The use of session pooling can reduce the number of individual sessions in memory. An effective session pool limits the number of sessions that can be used concurrently and employs a check-in/checkout system with the sessions. As sessions are freed up they can be checked-in and checked-out by other threads so as to limit the memory footprint on the app server. Sessions can be cleared using the **Clear** method and kept alive using the **KeepAlive** method. When all processing is done the sessions should be logged out, releasing them from memory.

Search requests are stateless calls and do not need to be tied to a specific session. One call after another can be made on the same session without the need to call **Clear** or **Logout**. Reuse of sessions in this case can reduce stress on the system and prevent unnecessary session usage.

Session Timeout

New Skies allows the session timeout to be defined by role. Care should be taken to determine the appropriate setting for each role so that sessions do not timeout too fast and do not remain idle too long. Applications that employ numerous concurrent logins should be given a short timeout to prevent the buildup of unintentionally abandoned sessions.

As a general rule session timeout should be between 15-20 mins for web users, 10 mins for anonymous users, and 5-10 mins for batch processes. It is recommended to employ separate roles in order to take advantage of these restrictions.

Stateful vs. Stateless

The New Skies API contains a mix of stateful and stateless methods.

Stateful Methods:

- Actions performed by the method effect the booking in state.
- The method will not require a Record Locator in the request.
- Operations performed by the method are not committed to the database until Commit/BookingCommit are explicitly called.
- Exception is Sell method which will hold the inventory until the session expires.

Stateless Methods:

- The method will require a Record Locator in the request to identify the booking to be updated.
- Operations will be committed to the database immediately

HTTP Compression

The New Skies web servers have been configured to return a compressed response when requested via the HTTP header. To ensure the highest performance, compression should be enabled via the request header.

Enabling Compression

Compression can be enabled by referencing the Navitaire.NPS.HTTPCompressionServices.dll assembly provided by Navitaire. After adding this assembly as a reference in your application, please add the following to you application/web configuration file:

```
<configuration>
<system.net>
<webRequestModules>
<remove prefix="http:"/>
<add prefix="http:" type="Navitaire.NPS.HTTPCompressionServices.CompressibleHttpRequestCreator,
Navitaire.NPS.HTTPCompressionServices" />
<remove prefix="https:"/>
<add prefix="https:" type="Navitaire.NPS.HTTPCompressionServices.CompressibleHttpRequestCreator,
Navitaire.NPS.HTTPCompressionServices" />
</webRequestModules>
</system.net>
</configuration>
```

Fiddler can be used to determine if the requests you are sending are being returned compressed. Check the response returned to see if the "GZIP Encoding" is selected:

Response is encoded and may require decoding before inspection. Click here to transform.

Transformer

Headers

TextView

ImageView

HexView

WebView

Auth

Caching

Cookies

Raw

JSON

XML

Response body: 2,349 bytes.

☐ Chunked Transfer-Encoding

Help...

HTTP Compression

☐ No Compression

☒ GZIP Encoding

☐ DEFLATE Encoding

☐ BZIP2 Encoding

☐ Use Zopfli to GZIP/DEFLATE

Original body was 9,174 bytes; compression saved 74.4%.

Compressor was set to fastest compression.

Filtered Methods

In an effort to reduce that amount of data returned in the XML response messages a number of filtered methods have been introduced. Where a filtered method is available these are considered the recommended call to use over the standard (non-filtered) method.

BookingManager

Available filtered methods:

- **GetBookingFromStateFiltered** – replaces the GetBookingFromState method for returning specific components of the current session state booking. **Note: This method does not return BookingSum, OperationsInfo, LegInfo, or LegSSR collections. If this data is needed, the standard GetBookingFromState method will need to be called.**
- **GetBookingStatelessFiltered** – this is the only method to retrieve a booking and not load it into session state.

Create Booking Flow

When creating bookings using the 3.4.x New Skies API, the following flow and suggestions should be followed.

Logon

Specify the ContractVersion that correlates to the version of New Skies you would like to execute as. Versioning goes to the 4th number of the New Skies version designator. This means that for the New Skies 3.4.12 release, you would specify 3412 in the ContractVersion property. If the ContractVersion is not specified the value is obtained from the New Skies app server corresponding to the current version of New Skies running. The ContractVersion should be set if there is concern about maintaining backwards compatibility after an upgrade.

Availability

Getting availability is recommended so that the correct fares can be sold. You are able to request availability for multiple flights in the same request.

GetAvailability

The **GetAvailability** call should **only** be used to retrieve a specific day's availability or a small date range (3-5 days). This is an expensive call and each day/O&D requested is counted as 1 look on your look-to-book report.

FareClassControl Filter

The "FareClassControl:Default" will return every fare for every flight that the logged in role can see. This is the largest and most resource intensive of the FareClassControl options. This call should be used extremely rarely and only if the specific case that all fares are needed for display or processing.

The "FareClassControl:LowestFareClass" will return only the lowest fare for each flight. This is the most common case and should be used as the default in most applications.

The "FareClassControl:CompressByProductClass" will return the lowest fare for each ProductClass specified in the availability request. This option should be used when fares from multiple product classes are desired.

The size of the response increases based on the number of days and fares included. Large responses can take additional time to transmit. **Excessive and improper availability requests are a leading cause of poor system performance.**

Price Itinerary

The GetItineraryPrice method is designed to provide a full quote of flights and services prior to selling. The response allows the client to see the full breakdown of fares, fees, charges, and taxes for display.

Sell (Journey)

The recommended method is the SellByJourneyKey option. For this you need to retrieve the JourneySellKey and the FareSellKey from the GetAvailability response for the flights you wish to sell. You are able to sell multiple flights in the same request.

UpdatePassengers

This method is intended to be used to update/replace all passenger information for the designated passenger number. You can update multiple passengers in the same request. **Data sent in this request for each passenger overwrites any previous data for that passenger. Any previous passenger data should be copied over to retain it.**

The following collections in the UpdatePassengers request are considered obsolete and should not be populated:

- **PassengerInfants** – the Infant singleton should be used instead.
- **PassengerInfos** – the PassengerInfo singleton should be used instead.
- **PassengerTypeInfo** – the PassengerTypeInfo singleton should be used instead.
- **PsuedoPassenger** – this property will be ignored.

Note: Data sent in this request for each passenger overwrites any previous data for that passenger. Any previous passenger data should be copied over to retain it.

AssignSeats

A single request can be used to assign seats on multiple segments. Refer to the Web Services Developer Guide for detailed information on properly using this method.

Sell (SSR)

This is another option of the Sell method. It is possible to sell multiple SSRs in the same request.

UpdateContacts

This method is intended to be used to add contact information to the booking. This was previously done during Commit, but has been broken out into its own method. This is available in 3.4.5 and higher.

AddPaymentToBooking

This method is used for the sole purpose of adding payment to the booking in session state.

BookingCommit

Calling this method with an empty request will commit the booking in session state to the database. Additional optional collections (booking contacts, booking comments, SourcePOS, and 3rd party record locators) can be added to the request to be submitted with the booking.

Logout

Once all processing is done, Logout should be explicitly called to release the session. Failure to explicitly logout will result in abandoned sessions that will reside on the app server until timeout. **Excessive abandoned sessions can result in resource constraints and performance degradation.**

Error Details:

Errors returned by API method

Below is a list of the errors that can be encountered when logging on to New Skies

Agent Locked

"The agent ([Domain]/[AgentName]) is locked."

Authorization

"Could not determine agent ([OrganizationCode]/[DomainCode]/[AgentName]) session settings."

Agent Authentication

"Session Token authentication failure."

No Agent Roles

"No roles specified for agent [OrganizationCode]/[DomainCode]/[AgentName]id [AgentID] in [SystemType][ChannelType]"

"Unable to find valid role for agent [OrganizationCode]/[DomainCode]/[AgentName]id [AgentID] in [SystemType][ChannelType]"

"Unable to find best role for agent [OrganizationCode]/[DomainCode]/[AgentName]id [AgentID] in [SystemType][ChannelType]"

Agent Authentication

"The agent ([DomainCode]/[AgentName]) was not authenticated."

Agent Not Found

"No agent found for requested agent name [DomainCode]/[AgentName]"

"No agent found for requested agent ID [AgentID]"

"The domain for the agent ([DomainCode]/[AgentName]) was not found."

"No organization found for requested agent name [DomainCode]/[AgentName]"

"No location found for requested agent name [DomainCode]/[AgentName]"

Agent Not Active

"The agent ([DomainCode]/[AgentName]) is not active."

Password Expired

"The agent ([DomainCode]/[AgentName]) must reset their password."

Password Expired

"The agent ([DomainCode]/[AgentName]) has an expired password."

Failed Logons

"The agent ([DomainCode]/[AgentName]) has too many failed logon attempts."

Session Timeout

"No such session [SessionID]."

"Session [SessionID] has expired."

Invalid Session

"The sessionState signature ([Signature]) is not equal to the provided signature ([Signature])"

Organization Not Found

"The Organization Code [OrganizationCode] does not exist"

Organization Not Active

"The Organization [OrganizationName]([OrganizationCode]) is not active."

Agent Access Restricted

"Agent {[DomainCode]/[AgentName]} with role [RoleCode] is not allowed to access the system."

The type of OSI for this instance. **Possible OSI Types:**

- Unknown : Unknown OSI type code.
- InvalidPriceStatus : Invalid price status.
- PromotionNotApplied : Promotion could not get applied on a given Journey.
- PaymentSystemDown : Communications with the payment system was unable to be established.

- `PaymentSystemException` : Communication with the payment system resulted in an exception condition
- `PaymentValidationError` : The payment did not conform to edits.
- `DistributeItineraryException` : An exception was thrown in post commit during itinerary distribution
- `FailedFareRestriction` : One of the fares requested failed the fare restriction check of one of the
- `DuplicateRecordLocatorSystemCode` : A record-locator was added with a record code and a system-code that already.
- `ArchivedBookingCommitted` : As a consequence only.
- `FailedToReinstateJourney` : While performing `UndoCancel` (reinstatement), a journey failed to sell.
- `FailedToReinstateSSR` : While performing `UndoCancel` (reinstatement), an SSR failed to sell.
- `FailedToReinstateSeat` : While performing `UndoCancel` (reinstatement), an Seat failed to sell.
- `AddBookingCommentsFailed` : While performing `AddBookingComments` the method had a problem.
- `AdditionalChargebackPaymentsRequired` : If a payment is chargebacked and has related transfers or refunds.
- `MinimumTimeBetweenJourneys` : If two journeys are too close between arrival of the first and departure.
- `CancelUnsupported` : Cancel unsupported in this state.
- `FailedSeatSSRValidation` : Seat-dependent SSRs did not pass validation.
- `FailedToRetrieveSSRFeePrice` : The call to `GetSSRFeePrice` was unsuccessful (no passenger exists on the booking).
- `SeatNotEquivalent` : Old and new seat assignments are not equivalent (numbers, properties, SSRs).
- `FailedToAssignAllSeats` : Failed to assign one or more seats.
- `FlightTimeOverlap` : Times of flights overlap each other.
- `UnpaidBalance` : Booking not fully paid (or payment not yet authorized).
- `MaxNumberOfPassengersExceeded` : Max. number of passengers exceeded.
- `TAFormOfPaymentIncluded` : TA form of payment included.
- `NoQualifyingSegmentsFound` : No qualifying segments.
- `NoPaymentsFound` : No qualifying segments.
- `MaxNumberOfCouponsPerPaxExceeded` : Maximum number of coupons per passenger exceeded.
- `CommitETicketFailed` : Failed to commit E-ticket/s.
- `UncommittedChangesDetected` : Uncommitted changes detected.
- `ExternalSystemError` : Uncommitted changes detected.
- `ETicketQueuingError` : Error in `ETicketQueuingSink`.
- `FailedToIssueNewETicket` : New Issue failure.
- `DisallowedActionStatusInBooking` : Disallowed Segment Status in booking.
- `ETicketingDisabled` : ETicketing is not enabled for the account.
- `SSRsUnavailable` : SSR is unavailable on the new flight.
- `CannotAddBaggageAllowanceAfterBagsAreChecked` : Cannot add Baggage Allowance after bags are checked.
- `BaggageProcessingFailed` : Baggage processing has failed.
- `HoldDateChanged` : The commit process has changed the hold date.
- `FailedToUnassignAllSeats` : Failed to un-assign one or more seats.
- `ExceededSeatAssignmentTimeLimit` : Exceeded the seat assignment time limit period.
- `FailedTravelDocValidation` : Failed Travel Doc Validation.
- `InvalidCollectedAmount` : The collected amount provided by the UI for an MCC payment is incorrect.
- `InvalidOperationOnCheckedPassenger` : Operation is not allowed on checked pax.
- `FewerSeatPreferencesMetOnNewSeat` : There are fewer seat preferences met on the new seat.
- `SSRNotSupportedOnNewSeat` : Seat-dependent SSR is not supported on the new seat.

- InvalidReferenceNumber : Reference Number is an invalid value.
- AgentActionNotPermitted : Agent does not have the required permissions.
- RecordNotFound : The desired record was not found.
- NoUpdateRequired : Update was not performed on specified record.
- ClearingCustomerProgram : Clearing Customer Program.
- SSRInfantAgeViolation : SSR is unavailable, infant age violation.